

Introdução a ML

João Eduardo Montandon
DCC024

ML

- Meta Language
 - Linguagem de propósito geral, alto nível, e funcional
 - Lançada em 1974 por Robin Milner (Universidade de Edinburgo)
 - Dedução de tipos (teorema Hindley-Milner)
 - Possui uma variedade de dialetos, SML-NJ é a mais conhecida
 - Influenciou linguagens como Clojure, F#, Haskell, OCaml, SML, etc
-

Setup

1. Ambiente local: [Standard ML of New Jersey \(smlnj.org\)](http://smlnj.org)
 2. Ambiente virtualizado
 1. Instalar docker
 2. Executar `$ docker run -it -v $HOME:$HOME -e SML_HISTORY=$HOME/.sml_history eldesh/smlnj`
-

Ambiente REPL

```
Standard ML of New Jersey
- 1 + 2 * 3;
val it = 7 : int;
- 1 + 2 * 3
= ;
val it = 7 : int
```

- Prompt disponível após `-`, ML vai responder o resultado e tipo da última operação
- Comandos terminados em ponto-e-vírgula (`;`)

- Variável `it` irá receber o valor da expressão que foi fornecida

Roteiro

1. Constantes
2. Operadores
3. Expressões Condicionais
4. Conversão
5. Variáveis
6. Tuplas e Listas
7. Funções
8. Tipos Explícitos

Constantes

Numéricos

```
- 1234;  
val it = 1234 : int  
- 123.4;  
val it = 123.4 : real  
- ~1234 + 1234;  
val it = 0 : int
```

- Tipos: `int` e `real`
- Use til (`~`) para representar um número negativo

Booleanos

```
- true;  
val it = true : bool  
- false;  
val it = false : bool  
- True;  
Error: unbound variable or constructor: True
```

- Tipo: `bool`
 - Valores são `true` ou `false`
 - ML é case-sensitive
-

Textuais

```
- "Joao\n";  
val it = "Joao\n" : string  
- "J";  
val it = "J" : string  
- #"J";  
val it = #"J" : char
```

- Tipos: `string` e `char`
 - Representadas com aspas duplas (" ")
 - caracteres devem ser precedidos por `#`
-

Operadores

Numéricos

```
- ~ 1 + 2 - 3 * 4 div 5 mod 6;  
val it = ~1 : int  
- ~ 1.0 + 2.0 - 3.0 * 4.0 / 5.0;  
val it = ~1.4 : real  
- 2 / 1;  
Error: operator and operand do not agree
```

- Operações suportadas para os dois tipos, exceto para divisão
 - Inteiro: `div` e `mod` | Real: `/`
 - Precedência `{+, -}` < `{*, /, div, mod}` < `{~}`
-

Strings

```
- "Linguagens" ^ "De" ^ "Programacao";  
val it = "LinguagensDeProgramacao" : string
```

- Concatenação: operador ^

Relacionais

```
- 2 < 3;  
val it = true : bool  
- 1.0 <= 1.0;  
val it = true : bool  
- #"d" > #"c";  
val it = true : bool  
- "abce" >= "abd";  
val it = false : bool
```

- Comparações de ordem (<, >, <=, >=) são aplicáveis a string, char, int e real
- Ordem em strings é lexicográfica

```
- 1 = 2;  
val it = false : bool  
- true <> false;  
val it = true : bool  
- 1.3 = 1.3;  
Error: operator and operand do not agree
```

- Comparações de igualdade: = e <>
- real não pode ser comparado por igualdade! (🤔)

```
- 1 < 2 orelse 3 > 4;  
val it = true : bool  
- 1 < 2 andalso not (3 < 4);  
val it = false : bool
```

- Operadores booleanos: andalso, orelse, e not
 - Operadores são *short-circuiting*
-

Precedência

Operadores
not, ~
*, /, div, mod
+, -, ^
<, >, <=, >=, =, <>
andalso
orelse

Expressões Condicionais

NÃO CONFUNDIR COM ESTRUTURAS CONDICIONAIS!

```
- if 1 < 2 then #"x" else #"y";  
val it = #"x" : char  
- if 1 > 2 then 34 else 56;  
val it = 56 : int  
- (if 1 < 2 then 34 else 56) + 1;  
val it = 35 : int
```

- Similar a um operador ternário!
- Não existe expressão `if ... then`, apenas!

```
<expressao-condicional> ::= if <expressao> then <express> else <expressao>;
```

- `<expressao>` do `if` precisa retornar booleano
- `<expressao>` do `then` e do `else` precisam ser do mesmo tipo! Por que? 🤔

Conversão De Tipos

```
- 1 * 2;
val it = 2 : int
- 1.0 * 2.0;
val it = 2.0 : real
- 1.0 * 2;
Error: operator and operand don't agree [overload - bad instantiation]
operator domain: real * real
operand:         real * 'Z[INT]
```

O que esse erro quer dizer? 🤔

-
- Os operadores são sobrecarregados para fazer uma operação com **pares de inteiros**, e outra com **pares de reais**
 - Problema ocorre quando há pares de tipos distintos
 - ML não faz conversão implícita
-

Função	Entrada	Saída
real	int	real
floor	real	int
ceil	real	int
round	real	int
trunc	real	int
ord	char	int
chr	int	char
str	char	string

```
- real(123);
val it = 123.0 : real
- floor(3.6);
val it = 3 : int
- floor 3.6; (* <--- The ML way! *)
val it = 3 : int
- str #"a";
val it = "a" : string
```

É válido? Qual o resultado?

- `floor(3.6)` ?
 - `floor 3.6` ?
 - `(floor) 3.6` ?
 - `(floor 3.6)` ?
 - `(floor) (3.6)` ?
 - `ceil 3.6 + 1` ?
 - `ceil floor 3.6` ?
-

Precedência Em Funções

- Chamadas de função são associativas a esquerda
 - Então `f a b` ➡ `(f a) b`
 - Além disso, funções tem altíssima precedência!
-

Definição De Variáveis

```
- val x = 1+2*3;
val x = 7 : int
- x;
val it = 7 : int
- val y = if x = 7 then 1.0 else 2.0;
val y = 1.0 : real
```

- `val` define uma nova variável
 - Nomes devem se iniciar com uma letra, seguida de 0+ letras, dígitos, ou underscore
-

Redefinição? 🤔

```
- val teste = 23;
val teste = 23 : int
- teste = 24;
val it = false : bool
- teste;
val it = 23 : int
```

```
- val fred = true;
val fred = true : bool
```

- Variáveis não podem ser redefinidas
 - **Para toda definição** cria-se um novo espaço para armazenar o valor
-

Curiosidade sobre `val it = ....`

- O retorno do input é armazenada em uma variável especial, chamada `it`
 - Se fornecer apenas `- 1 + 2;`, é como se estivéssemos digitado `val it = 1 + 2;`
-

Tuplas

```
- val barney = (1+2, 3.0*4.0, "brown");
val barney = (3,12.0,"brown") : int * real * string
- val point1 = ("red", (300,200));
val point1 = ("red",(300,200)) : string * (int * int)
- #2 barney;
val it = 12.0 : real
- #1 (#2 point1);
val it = 300 : int
```

- Formadas a partir de valores entre parenteses
 - Pode conter outras tuplas
 - Para recuperar um elemento da dupla, utilize `#posicao tupla`
-

Construtor De Tipos

- Tipo de uma tupla é construído a partir de seus elementos
- A tupla `(1+2, 3.0*4.0, "brown")` tem como tipo `int * real * string`
- A tupla `("red", (300,200))` tem como tipo `string * (int * int)`

Construtor de tipos: Permite criar novos tipos a partir dos existentes

```
- (1, 2);  
val it = (1,2) : int * int  
- (1);  
val it = 1 : int  
- #1 (1);  
Error: operator and operand don't agree
```

- Tuplas tem pelo menos dois elementos
- Parenteses com um elemento serve para agrupar operações

Listas

```
- [1,2,3];  
val it = [1,2,3] : int list  
- [true];  
val it = [true] : bool list  
- [(1,2),(1,3)];  
val it = [(1,2),(1,3)] : (int * int) list  
- [[1,2,3],[1,2]];  
val it = [[1,2,3],[1,2]] : int list list
```

- Formada a partir de valores entre colchetes
- Todos os elementos **precisam** ser do mesmo tipo
- Tipo `x list` também é um construtor de tipos!

Tuplas X Listas

```
- val x = (1, 2, 3);  
val x = (1,2,3) : int * int * int  
- val y = [1, 2, 3];  
val y = [1,2,3] : int list
```

- Tuplas: **exatamente** três elementos do tipo `int`
- Listas: uma lista de `int`

Quando utilizar cada um? 🤔

```
- [];  
val it = [] : 'a list  
- nil;  
val it = [] : 'a list
```

- Lista vazia representada por `[]` ou `nil`
 - `'a list` ➡ lista de elementos de tipo desconhecido
-

A Função `null`

```
- null [];  
val it = true : bool  
- null [1,2,3];  
val it = false : bool
```

- `null` verificar se uma lista é vazia
 - Também pode ser feito por `lista = []` ⚠
-

Concatenação

```
- [1,2,3] @ [4,5,6];  
val it = [1,2,3,4,5,6] : int list  
- [1] @ [];  
val it = [1] : int list  
- 1 @ [];  
Error: operator and operand do not agree
```

- `list @ list` ➡ concatena duas listas
 - As listas precisam ser do mesmo tipo!
-

Operador `::`

```
- val x = #"P" :: [];  
val x = [#"P"] : char list  
- val y = #"L" :: x;  
val y = [#"L",#"P"] : char list
```

- "Construir" uma lista a partir de elementos existentes
 - Input: um elemento de um tipo, e uma lista do mesmo tipo
 - Output: uma **nova lista** contendo elemento na frente da lista antiga
 - Na prática, enfileira elementos no início da lista
 - **Associativo à direita!**
-

```
- val list = 1::2::3::4::[];
val list = [1,2,3,4] : int list
- hd list;
val it = 1 : int
- tl list;
val it = [2,3,4] : int list
- hd (tl list);
val it = 2 : int
- tl (tl (tl (tl list)));
val it = [] : int list
```

- `hd` ➡ retorna elemento da frente da lista
 - `tl` ➡ retorna restante da lista
-

```
- val list = explode "Linguagens";
val list = ["L",#"i",#"n",#"g",#"u",#"a",#"g",#"e",#"n",#"s"] : char list
- implode list;
val it = "Linguagens" : string
```

- `explode` ➡ converte string em vetor de caracteres
 - `implode` ➡ converte vetor de caracteres em strings
-

Definição De Funções

```
- fun firstChar s = hd (explode s);
val firstChar = fn : string -> char
- firstChar "abc";
val it = #"a" : char
```

Coisas muito interessantes aconteceram aqui! 🙄

- Funções definidas a partir de `fun`
- Devem sempre retornar algo
- ML Inferiu o tipo de entrada e saída! ^[1]
 - Se `s` é parâmetro de `explode`, então deve ser do tipo `string`
 - Se `hd` é aplicado sobre um `char list` então retorna um `char`

```
<fun-def> ::= fun <nome-funcao> <parametro> = <expressao> ;
```

- Parâmetro... apenas um? 🙄
- Expressão... apenas um? 🙄

Construtor De Tipos

```
- firstChar;  
val it = fn : string -> char
```

- Para função, um novo tipo é construído envolvendo sua entrada e saída
- `tipo_input -> tipo_output`
- Exemplo: `firstChar` recebe uma `string` e retorna um `char`

```
- fun quot(a, b) = a div b;  
val quot = fn : int * int -> int  
- quot(6,2);  
val it = 3 : int
```

Quantos parâmetros? 🙄

```
- val par = (6,2);  
val par = (6,2) : int * int
```

```
- quot par;  
val it = 3 : int
```

- Funções aceitam apenas **um parâmetro**
- Mais parâmetros? **Tuplas!**

Recursividade

```
- fun fact n = if n = 0 then 1 else n * fact(n-1);  
val fact = fn : int -> int  
- fact 5;  
val it = 120 : int
```

- Caso base: `n` é zero
- Passo recursivo: `n * fact(n-1)`

Fundamental para linguagens funcionais!

```
- fun listsum l =  
= if null l then 0  
= else hd l + listsum(tl l);  
val listsum = fn : int list -> int  
- listsum [1,2,3,4,5,6,7,8,9,10];  
val it = 55 : int  
- listsum [1.0, 2.0];  
Error: operator and operand do not agree
```

Qual é o caso base e o passo recursivo? 🤔

```
- fun size l =  
= if null l then 0  
= else 1 + length(tl l);  
val size = fn : 'a list -> int  
- size [1,2,3,4,5];  
val it = 5 : int  
- size (explode "0 rato roeu a roupa do rei de roma");  
val it = 34 : int
```

Qual tipo de lista essa função aceita? 🤔

```
- fun length l =  
= if l = [] then 0  
= else 1 + length(tl l);  
val length = fn : 'a list -> int  
- length [1,2,3,4];  
val it = 4 : int  
- length (explode "o rato roeu a roupa do rei de roma");  
val it = 34 : int  
- length [1.0, 2.0, 3.0];  
Error: operator and operand do not agree [equality type required]
```

Qual tipo de lista essa função aceita? 🤔

```
- fun reverse l =  
= if null l then nil  
= else reverse(tl l) @ [hd l];  
val reverse = fn : 'a list -> 'a list  
- reverse [1,2,3];  
val it = [3,2,1] : int list
```

Qual é o caso base? E o passo recursivo?

Tipos Explícitos

- Tipos primitivos: `int`, `real`, `bool`, `char`, `string`
- Construtores de tipo: `*` (tuplas), `list` (listas), `->` (funções)
 - Precedência: `list` $\rightarrow$ `*` $\rightarrow$ `->`
 - `int * bool list = int * (bool list)`
 - `int * bool list -> real = (int * (bool list)) -> real`

```
- fun prod(a, b) = a * b;  
val prod = fn : int * int -> int
```

Por que `int`? 🤔

Anotação De Tipos

- Cada operador tem um tipo-padrão associado a ele
 - `*`, `+`, `-` → `int`
 - `/` → `real`
- Solução: anotar o tipo de uma variável explicitamente, se necessário
- Pode aparecer após qualquer variável ou expressão.

```
- fun prod(a:real, b:real) : real = a * b;  
val prod = fn : real * real -> real
```

Declarações equivalentes (por que?)

```
fun prod(a,b):real = a * b;  
fun prod(a:real,b) = a * b;  
fun prod(a,b:real) = a * b;  
fun prod(a,b) = (a:real) * b;  
fun prod(a,b) = a * b:real;  
fun prod(a,b) = (a*b):real;  
fun prod((a,b):real * real) = a*b;
```

Conclusões

- ML: alto-nível, propósito geral, e **funcional**
- tipos primitivos: `int`, `real`, `bool`, `char`, `string`
- Operadores: `~`, `+`, `-`, `*`, `div`, `mod`, `/`, `^`, `::`, `@`, `<`, `>`, `<=`, `>=`, `=`, `<>`, `not`, `andalso`, `orelse`
- Condicionais são expressões!
- Funções pré-definidas `real`, `floor`, `ceil`, `round`, `trunc`, `ord`, `chr`, `str`, `hd`, `tl`, `explode`, `implode`, `null`

-
- Novas variáveis: `val`
 - Tuplas: `(a, b, c, d)`
 - Listas: `[a, b, c, d]`
 - Construtores de Tipos (`*`, `list`, `->`)

- Declaração de funções: `fun` , único argumento, tipo inferido pela linguagem, recursividade!
 - Anotações de tipo, caso necessário
-
-

1. [Hindley–Milner type system - Wikipedia](#) ↩